The Journal of Open Source Software

pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi [1¶], **Arnaud Delorme** [1,2], **and Scott Makeig** [1]

1 Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA **2** Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Independent Component Analysis (ICA) is a widely applied method for separating electroencephalographic and magnetoencephalographic (EEG/MEG) recordings into maximally independent sources that isolate brain, muscle, and artifact activities for downstream analysis (Iversen & Makeig, 2019; Makeig et al., 1995; Vigário et al., 1997). Adaptive Mixture ICA (AMICA) (Palmer et al., 2012) generalizes single-model ICA to a mixture of such models with adaptive source densities, and produces both the least dependent and the most dipolar (and thus most physiologically interpretable) component decompositions of EEG among the algorithms benchmarked by Delorme et al. (2012). Its reference implementation, written by Jason Palmer, is a Fortran program parallelized with the Message Passing Interface (MPI) and distributed as a compiled binary callable from MATLAB/EEGLAB, which is difficult to install, runs only on the central processing unit (CPU), and is not usable from a Python scientific workflow.

`pamica` is a Python implementation of AMICA that reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA graphics processing units (GPUs, via CUDA), and Apple GPUs (Apple's MLX array framework (Hannun et al., 2023)). It is a complete reimplementation built on PyTorch (Paszke et al., 2019), NumPy (Harris et al., 2020), and SciPy (Virtanen et al., 2020), not a wrapper around the Fortran binary, and exposes a scikit-learn-style estimator under a BSD-3-Clause license. In double precision it reproduces the reference score-function algebra to floating-point round-off; it also runs in single precision (float32), unavailable in the CPU-only binary and required for Apple GPUs, which have no float64 and host the fastest backend (MLX). `pamica` writes output in the binary format that EEGLAB's AMICA loader reads: a single-model output file is byte-identical in layout to a native AMICA file, and multi-model output round-trips through the same loader. Correctness is defined as parity with the Fortran reference for the single-model case and, because multi-model AMICA is not partition-identifiable, as a similar distribution of solutions for the multi-model case; both are validated on real EEG against the reference binary. The software is at https://github.com/sccn/pAMICA (archived at doi:10.5281/zenodo.21312148).

Statement of need

AMICA decompositions of neuroelectromagnetic data are well suited to equivalent-dipole source localization and automated component classification (Pion-Tonachini et al., 2019). Yet its reference implementation is MATLAB-only Fortran, an increasing obstacle as neuroimaging analysis moves toward Python, for example MNE-Python (Gramfort et al., 2013): an AMICA

that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically, is needed for modern pipelines.

General-purpose Python ICA implementations do not fill this gap. `scikit-learn` and `MNE-Python` provide FastICA (Hyvärinen & Oja, 2000) and Infomax (Bell & Sejnowski, 1995; Lee et al., 1999), while Picard (Ablin et al., 2018) offers faster-converging maximum-likelihood ICA; none implement AMICA's mixture of models, adaptive generalized-Gaussian source densities, or Newton updates, so they do not reproduce AMICA decompositions. `pamica` targets EEG and MEG analysts who want AMICA-quality decompositions inside a Python pipeline, users of GPU hardware who want faster runs than the CPU-only binary, and methodologists who need a transparent reference implementation to build on.

Implementation and validation

`pamica` provides a natural-gradient (Amari, 1998) expectation-maximization (EM) backend that ports the full AMICA algorithm: exact-EM mixture updates, a positive-definite Newton step (Palmer et al., 2008), symmetric zero-phase-component-analysis (ZCA) sphering, the five source-density families of the reference (generalized Gaussian, Gaussian, logistic, sub-Gaussian, and the extended-Infomax kurtosis switcher), a mixture of ICA models, and component sharing across models. It also computes mutual information reduction (MIR) and pairwise mutual information (PMI), separation-quality metrics useful for benchmarking ICA algorithms (Delorme et al., 2012; Frank et al., 2023).

`pamica`'s conformity with the reference binary is measured with two complementary metrics: Hungarian-matched component correlation and the Amari distance (Amari et al., 1996), a relabeling- and scale-invariant unmixing-matrix metric that needs no assignment step. Both implementations were run for the AMICA heuristic default of 2000 iterations with Newton off (`do_newton=0`) and otherwise-default parameters (settings transcribed between `pamica`'s JSON and Fortran's native text format). This 2000-iteration budget is a practical stopping point, not a convergence point: source separation keeps improving slowly at higher iteration counts (Frank et al., 2023). Newton acceleration is disabled here to isolate the algorithm from initialization: the Newton update speeds convergence, but once enabled it lets independently seeded runs settle a few under-determined components into different, equally likely optima, whereas a matched initialization recovers agreement (documentation). The single-model comparison uses a well-determined external recording (OpenNeuro ds002718, $k \approx 153$, where $k =$ frames over squared channel count (Frank et al., 2025)), well past the ~60 threshold where cross-backend agreement plateaus, together with the bundled 32-channel sample ($k \approx 30$); Table 1 gives each metric's dataset. Score functions and per-block sufficient statistics are exact to floating-point resolution against the literal Fortran expressions on the bundled sample. A mixture of ICA models is not partition-identifiable, so exact partition parity is the wrong bar for the multi-model case; it is instead assessed by whether the two implementations sample a similar distribution of solutions, across ensembles of 20 runs each (Figure 1). A permutation test finds no evidence that cross-implementation agreement is worse than Fortran's own run-to-run agreement. Multi-model log-likelihood distributions still differ slightly at a matched iteration budget (`pamica` needs about twice as many iterations to reach Fortran's mean), so full-likelihood similarity is not yet claimed.

Table 1: Parity of `pamica` with the Fortran reference. The two single-model conformity metrics use different recordings, hence the two data-adequacy ratios k: the correlation headline uses a well-determined external recording ($k \approx 153$), while the Amari distance and score-function checks use the bundled 32-channel sample ($k \approx 30$). Multi-model agreement is distributional, since a mixture of models is not partition-identifiable; the ensemble row is the mean difference between cross-implementation and within-Fortran agreement, with a run-level permutation p-value. Values are means (sd, standard deviation) over matched components or, for multi-model, over within/cross-implementation run pairs (190/400).

Regime	Metric (dataset)	Result (mean)
Single-model	Log-likelihood gap to Fortran (ds002718, $k \approx 153$)	within ~0.0005 of -3.6993
Single-model	Hungarian-matched component correlation (ds002718, $k \approx 153$)	0.998
Single-model	Amari distance (bundled 32-channel sample, $k \approx 30$)	0.006
Single-model	Score functions and sufficient statistics (bundled sample)	exact, $\sim 10^{-15}$
Multi-model	Component correlation, single run: `pamica`-Fortran; Fortran-Fortran (bundled)	0.65; 0.64 (sd 0.05)
Multi-model	Amari distance, single run: `pamica`-Fortran; Fortran-Fortran (bundled)	0.163; 0.174 (sd 0.02)
Multi-model	Ensemble agreement, cross-implementation − within-Fortran, 20 runs each (bundled)	correlation $+0.011$ ($p = 0.96$); Amari -0.011 ($p > 0.999$)

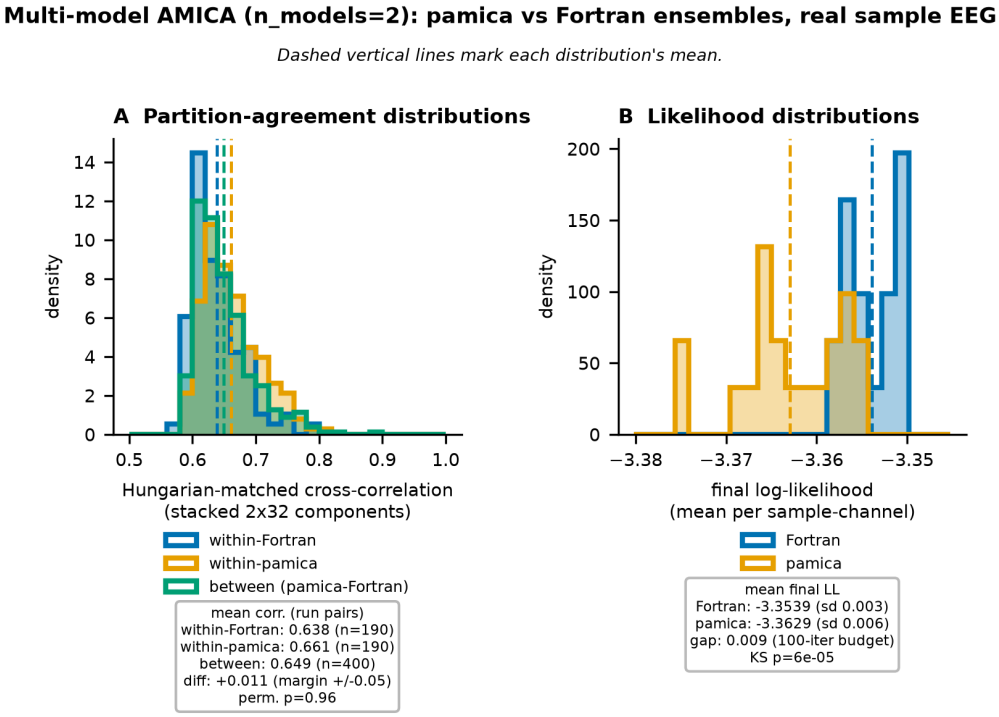

Figure 1: Multi-model solution-ensemble partition-correlation distributions (panel A) and log-likelihood distributions (panel B) for 20 `pamica` and 20 Fortran fits of the sample EEG; dashed lines mark each distribution's mean. The within-Fortran, within-`pamica`, and between-implementation correlation distributions overlap, so the single-run correlation reflects the estimator's intrinsic run-to-run spread rather than a gap to the reference. Panel B's apparent separation is a 0.009 log-likelihood gap on a ~0.035 axis.

All backends converge to the same single-model log-likelihood on real EEG (maximum pairwise difference ~0.003). On Apple Silicon, MLX is the fastest backend and flat with channel count (Table 2); PyTorch-MPS is never a win. Double-precision CUDA is the reproducible NVIDIA path; native Fortran scales with CPU cores and, with enough cores pinned, can beat the GPU on a larger, hotter host, though it does not match Apple's MLX on laptop hardware. A data-size sweep (documentation) shows cross-backend component agreement rising with frames per channel and plateauing near 0.98 once the decomposition is well-determined, where two independent double-precision implementations agree at a mean of 0.995; single-precision runs agree with float64 to four to five significant digits, so float64 stays the default for parity.

Table 2: Single-model throughput on real 70-channel EEG (`n_mix`=3, `pdftype`=0, `block_size`=512; warm, minimum of repeated runs). CPU, MPS, and MLX on Apple Silicon; CUDA on a separate NVIDIA RTX 4090 (float32 comparable, ~36 ms). The two native-Fortran rows are from a separate core-count sweep (documentation) at each backend's plateau; the other CPU rows use platform-default threads and are not core-matched to Fortran. Unlike the correctness comparison, this benchmark uses external data (OpenNeuro ds002718, one subject so far) and specific GPU hardware.

Backend (device)	Precision	ms / iteration
MLX (Apple GPU)	float32	25
CUDA (NVIDIA RTX 4090)	float64	39
Native Fortran (Intel Core i9-13900K, 24 cores)	float64	30
PyTorch CPU (Apple Silicon)	float64	193
Native Fortran (Apple Silicon, 8 cores)	float64	70
PyTorch MPS (Apple GPU)	float32	255

Backend (device)	Precision	ms / iteration
NumPy (reference, Apple Silicon)	float64	622

The correctness harness never uses synthetic data; the multi-model and score-function checks need no external download (bundled sample only). The full performance tables, per-run Amari-distance detail, data-size sweep, and step-by-step reproduction commands are in the documentation.

State of the field

`pamica` complements rather than replaces the reference Fortran AMICA used with EEGLAB (Delorme & Makeig, 2004): it uses the same output format as that Fortran version, adds a Python API with GPU support, and runs the reference Fortran itself through a bundled dependency-free native build (no Intel Math Kernel Library or MPI runtime). Two other Python AMICA reimplementations have appeared (Esmaeili, 2025; Herforth, 2026), both of which provide MNE-Python-compatible objects; `pamica` offers a scikit-learn-style array API, byte-identical EEGLAB I/O, an MLX backend for Apple GPUs, and an optional MNE-Python wrapper. What sets `pamica` apart is the depth of its Fortran-parity validation (Table 1): source-density score functions bit-exact to floating-point resolution against the literal Fortran expressions, and a distributional-similarity framework for the non-identifiable multi-model case.

Acknowledgements

We thank Jason Palmer and his advisor Ken Kreutz-Delgado, co-developers of AMICA, for the reference implementation. We also thank the EEGLAB community for the tools and sample data used to validate this work. Two of the authors are original developers of the methods `pamica` builds on: S.M. co-developed the AMICA algorithm (Palmer et al., 2012) and A.D. is a lead developer of EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of Health grant R01-NS047293 (to A.D. and S.M.).

References

Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*, *66*(15), 4040–4053. https://doi.org/10.1109/TSP.2018.2844203

Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, *10*(2), 251–276. https://doi.org/10.1162/089976698300017746

Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal separation. *Advances in Neural Information Processing Systems*, *8*, 757–763.

Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind separation and blind deconvolution. *Neural Computation*, *7*(6), 1129–1159. https://doi.org/10.1162/neco.1995.7.6.1129

Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of Neuroscience Methods*, *134*(1), 9–21. https://doi.org/10.1016/j.jneumeth.2003.10.009

Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG sources are dipolar. *PLoS ONE*, *7*(2), e30135. https://doi.org/10.1371/journal.pone.0030135

Esmaeili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and scientific EEG workflows*. https://github.com/snesmaeili/amica.

Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2023). An exploration of optimal parameters for efficient blind source separation of EEG recordings using AMICA. *2023 IEEE 23rd International Conference on Bioinformatics and Bioengineering (BIBE)*, 205–210. https://doi.org/10.1109/BIBE60311.2023.00040

Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2025). Quantitative exploration of sufficient data quantities for EEG decomposition using AMICA. *2025 12th International IEEE/EMBS Conference on Neural Engineering (NER)*, 532–536. https://doi.org/10.1109/NER61569.2025.11588993

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, *7*, 267. https://doi.org/10.3389/fnins.2013.00267

Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on apple silicon*. https://github.com/ml-explore/mlx.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component analysis*. https://github.com/DerAndereJohannes/pyamica.

Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications. *Neural Networks*, *13*(4-5), 411–430. https://doi.org/10.1016/S0893-6080(00)00026-5

Iversen, J. R., & Makeig, S. (2019). MEG/EEG data analysis using EEGLAB. In *Magnetoencephalography: From signals to dynamic cortical networks* (pp. 391–406). Springer International Publishing.

Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural Computation*, *11*(2), 417–441. https://doi.org/10.1162/089976699300016719

Makeig, S., Bell, A. J., Jung, T.-P., & Sejnowski, T. J. (1995). Independent component analysis of electroencephalographic data. *Advances in Neural Information Processing Systems*, *8*, 145–151.

Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). *AMICA: An adaptive mixture of independent component analyzers with shared components*. Swartz Center for Computational Neuroscience, University of California San Diego. https://sccn.ucsd.edu/~jason/amica_a.pdf

Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 1805–1808. https://doi.org/10.1109/ICASSP.2008.4517982

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*, 8024–8035.

Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated electroencephalographic independent component classifier, dataset, and website. *NeuroImage*, *198*, 181–197. https://doi.org/10.1016/j.neuroimage.2019.05.026

Vigário, R., Jousmäki, V., Hämäläinen, M., Hari, R., & Oja, E. (1997). Independent component

analysis for identification of artifacts in magnetoencephalographic recordings. *Advances in Neural Information Processing Systems*, *10*, 229–235.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2